CS 334 : COMPILER DESIGN LABORATORY **Scheduled : March 17 to 19, 2026**
Premises : Lab 1, 3 and 6

Faculty : Bikas Kanti Sarkar, Indrajit Mukherjee, Prashant Pranab, Anup Keshri and Supratim Biswas
Teaching Assistant : Swarna Aishwarya Twinkle (Research Scholar)

Lab Assignment 7 : Processing declarations in a single scope, symbol table and relevant semantic analysis.

General Remarks :

1. Grammar for declaration of scalar variables, creation of a desirable symbol table structure to save the declarations in an input file; and semantic analysis that detects few errors in declarations are the focus of this assignment. Two grammars, G1 and G2, are given in "Lab7handout.pdf". On completion of the experiment, you should be able to comment on which grammar should be chosen and why.

2. The sizes in bytes of basic data types on your server can be obtained by writing a C program. On my laptop the sizes are as follows.

char	short	int	long	long long	float	double	long double
1	2	4	8	8	4	8	16

You may write a C program to determine the sizes on your server and use them to write your semantic actions.

P1. The objective of this problem is to design a declaration parser using the grammar G1. Read the lex and yacc scripts, “declbasic.l” and “declbasic-incomplete.y”, the sample input and output file, "declinp1.txt" and "declout1.txt" respectively. Make necessary changes, as appropriate, so that the output of your parser is same as the given output. Test your parser with two inputs created by you, one without a semantic error and another with a semantic error.

Resources Given : declbasic.l declbasic-incomplete.y declinp1.txt declout1.txt

P2. The declaration parser of P1 is not capable of detecting semantic errors in declarations, unless it is changed significantly. An essential requirement for detecting semantic errors in declarations is to save all the attributes of a declaration in some data structure D, and then use D to check the semantic validity of the declarations and also their use. A hash table is known to be an efficient choice for D, however in our experimentation, we shall use a simple unsorted table for ease of implementation. The structure of the symbol table, is explained in "Lab7handout.pdf".

Consider a main() function given in the file, "symtab-main.c", which in turn uses a symbol table data structure defined in the file, "symtab-basic-functions.c". You are also given an input file, "idlist.txt".
Perform the following operations : $ gcc symtab-main.c -o symtab $ ./symtab idlist.txt
Observe the generated output and get familiar with the simple symbol table structure and its use.

P3. Your task is to design a declaration parser with semantic analysis that can process a set of input declarations Sample input and output for this problem are given for your reference. You may adopt the given code in P2 or write your own code for the symbol data structure.

Sample input	**Desired Output**
int a, b, a; char ch1; float fl1, ch1;	Error: redeclaration of ‘a’ at index [1] Error: conflicting types for ‘ch1' as ‘float’ previous declaration of ‘ch1’ with type ‘char’ at index [3] Index[0] Symtab Global 13 4 Index [1] a int 4 0 Index [2] b int 4 4 Index [3] ch1 char 1 8 Index [4] fl1 float 4 12

P4. Design a new parser using grammar G2. The symbol table structure remains the same as that of P3. The difference in P4 is that all identifiers in a single declaration statement are first collected into a list and the list appended to the symbol table in a single call to append(). You would need to define a list data structure for this solution. You may write your own list data structure or examine the 2 files, "list-main.c" and "list-functions.c" and customize them to suit your requirements.

End of the Experiment 7